



**Politecnico
di Torino**

N-MON

CRITICAL DESIGN REVIEW

Presented by Group - G

Braia Lorenzo Eustachio - s346316@studenti.polito.it

Cavallaro Lorenzo - s346742@studenti.polito.it

Peradotto Simone - s343420@studenti.polito.it



OVERVIEW

- Objectives
- PDR open points
- Architecture
- WBS
- Gantt Chart

OBJECTIVES

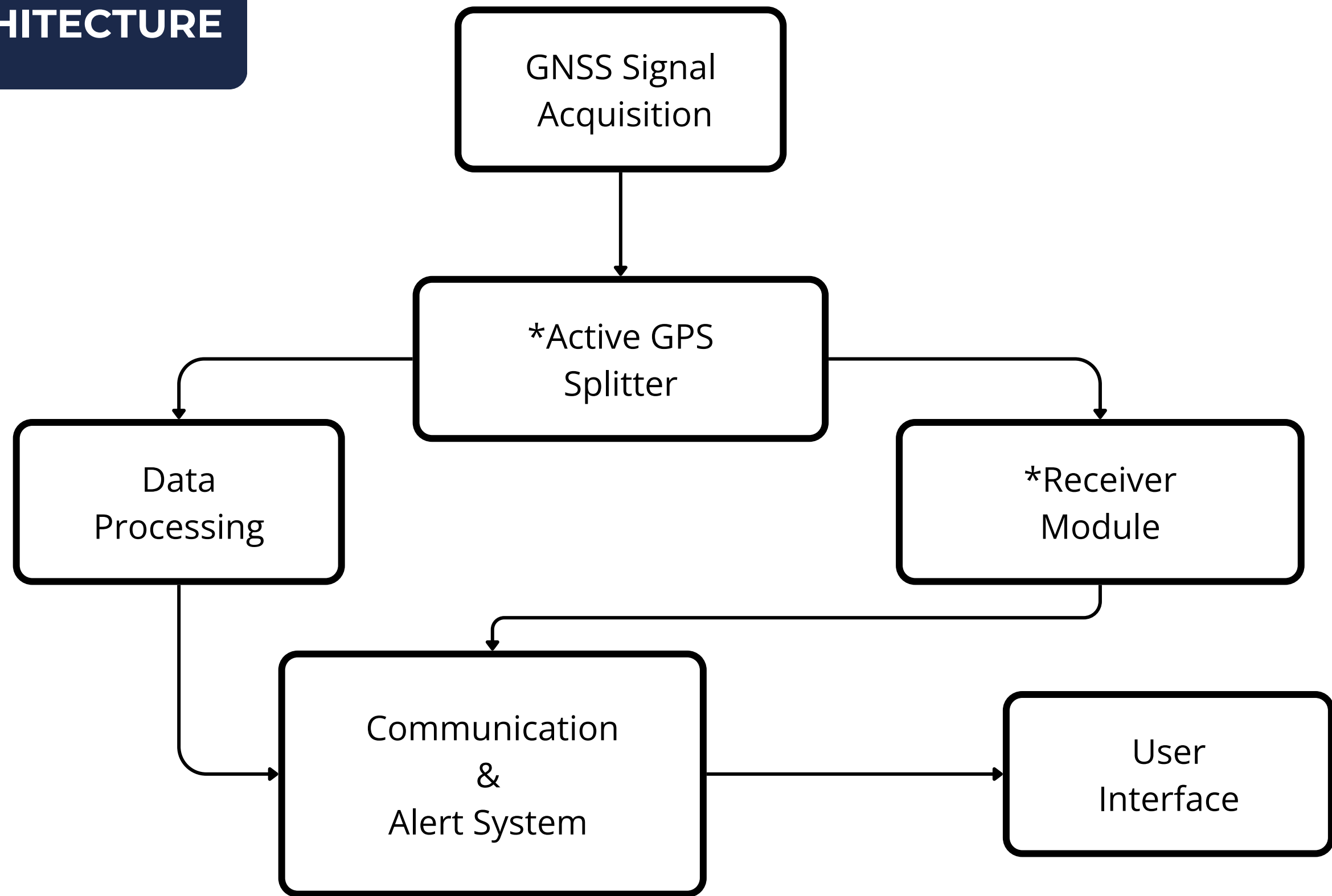
Main Goal

- To develop a compact and cost-effective monitoring system that detects and classifies GNSS jamming interference in real-time using Artificial Intelligence Algorithms in order to protect high-value assets during transit.
- To provide an autonomous system that enhances the security of fleet management by immediately alerting both stakeholders and drivers, of jamming events, allowing for rapid response.

PDR OPEN POINTS

- Processing Unit Selection : Raspberry Pi 5 - 8GB vs NVIDIA Jetson Orion Nano
- AI Algorithm Selection : Convolutional Neural Network vs Recurrent Neural Network
- Communication Protocol : HTTP vs MQTT
- Control Center UI feedback : adding selection filters and map visualization of jamming events

HIGH-LEVEL ARCHITECTURE



*change from PDR architecture

PATCH ANTENNA

Model GPS P1MAM

Technical Specification

- Operating Frequency: T1 1575.42 ± 1.023 MHz
- Bandwidth: 10 MHz with $S_{11} \leq -10$ dB (90% of signal is acquired)
- Gain: 28 dB
- Filter: DR Filter to avoid interference from signals in the near spectrum
- Power Supply: 2.3 - 5.5 V ideal for trucks



ACTIVE SPLITTER

A passive splitter introduces ~3.5 dB insertion loss on each port an active splitter (which has a built-in amplifier) instead compensates for this. It is critical since we're already dealing with the weak GNSS signal level.

Satisfied requirements:

UREQ_1 , UREQ_2 , UREQ_3 , FREQ_1 , FREQ_2 , FREQ_3 , FREQ_4 , FREQ_5 , TREQ_1 , TREQ_2 ,
TREQ_3 , TREQ_4 , TREQ_5 , TREQ_6 , TREQ_7 , TREQ_8 , TREQ_9

RECEIVER MODULE

Main goal

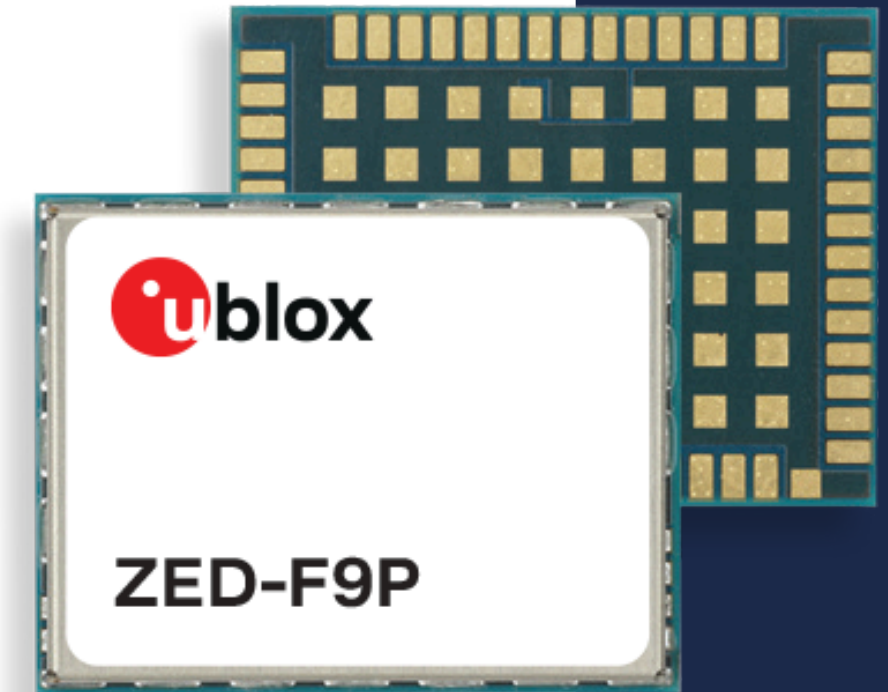
- saving and showing in the UI the last position of the truck before the jamming detection

Model

- ZED-F9P module

Technical Specification

- GNSS constellation: GPS / QZSS, GLONASS, Galileo, BeiDou
- GNSS bands: L1C/A, L2C, L1OF, L2OF, E1B/C, E5b, B1I, B2I
- Interfaces: UART x2, USB
- Power Supply: 2.7 - 3.6 V ideal for trucks
- Temperature range [°C]: -40, +85



Satisfied requirements:

UREQ_1 , UREQ_2 , UREQ_3 , **FREQ_1** , FREQ_2 , FREQ_3 , FREQ_4 , FREQ_5 , **TREQ_1** , TREQ_2 ,
TREQ_3 , TREQ_4 , TREQ_5 , TREQ_6 , TREQ_7 , TREQ_8 , TREQ_9

HACKRF ONE

Open-Source SDR Platform

- Full documentation and active community support and compatible with GNU Radio ecosystem.
Extensive software libraries available

Wide Frequency Coverage

- Operating range: 1 MHz to 6 GHz , covers al GNSS bands so in the future can be extended to monitor L2/L5

Technical Specifications

- Sampling rate: Up to 20 MS/s and ADC resolution of 8-bit

Cost-Effective

- Price: ~ 300€ (2000€+ for professional SDR equipment)

Low Power Consumption

USB powered: ~2.5W maximum consumption, so suitable for vehicle deployment without dedicated power supply



Satisfied requirements:

UREQ_1 , UREQ_2 , UREQ_3 , **FREQ_1** , **FREQ_2** , FREQ_3 , FREQ_4 , FREQ_5 , **TREQ_1** , **TREQ_2** ,
TREQ_3 , TREQ_4 , **TREQ_5** , TREQ_6 , TREQ_7 , **TREQ_8** , TREQ_9

PROCESSING UNIT

Model NVIDIA Jetson Orion Nano

Technical Specification

- Architecture : SoM + Carrier Board
- 6-Core CPU + GPU (40 TOPS)
- 8 GB RAM
- Storage : NVMe
- Price : ~350€
- Size : 100mm x 79mm x 21mm



Satisfied requirements:

UREQ_1 , UREQ_2 , UREQ_3 , **FREQ_1 , FREQ_2 , FREQ_3 , FREQ_4 , FREQ_5 , TREQ_1 , TREQ_2 , TREQ_3 , TREQ_4 , TREQ_5 , TREQ_6 , TREQ_7 , TREQ_8 , TREQ_9**

DATA PROCESSING



1) Signal Acquisition

- HackRF One continuously captures raw IQ samples from L1 GNSS band
- Signal segments: 100 μ s snapshots

2) Short-Time Fourier Transform (STFT)

- Converts time-domain IQ samples into time-frequency representation (spectrogram)
- Output: 2D matrix capturing spectral evolution over time

3) AI Model Inference

- Normalized spectrogram fed directly into AI Algorithm network
- Output: classification across 16 classes (5 jamming types [LWF, LN, TRI, TRIW, TICK] per 3 JSR levels [LOW, MID, HIGH] + clean signal)

4) Decision & Alert

- If jamming detected trigger MQTT alert

Satisfied requirements:

UREQ_1 , UREQ_2 , UREQ_3 , FREQ_1 , FREQ_2 , FREQ_3 , FREQ_4 , FREQ_5 , TREQ_1 , TREQ_2 ,
TREQ_3 , TREQ_4 , TREQ_5 , TREQ_6 , TREQ_7 , TREQ_8 , TREQ_9

CNN MODEL

Why CNN over GRU?

- 2D spectrograms → CNNs excel at **spatial feature extraction**
- **Transfer learning** from ImageNet (pre-trained features)
- **Hardware optimized**: CNNs leverage NVIDIA GPU parallelism more efficiently than sequential GRUs

1) Model Overview

- **ResNet-18** with **Transfer Learning** adapted for single channel spectrogram analysis.

2) Dataset Structure

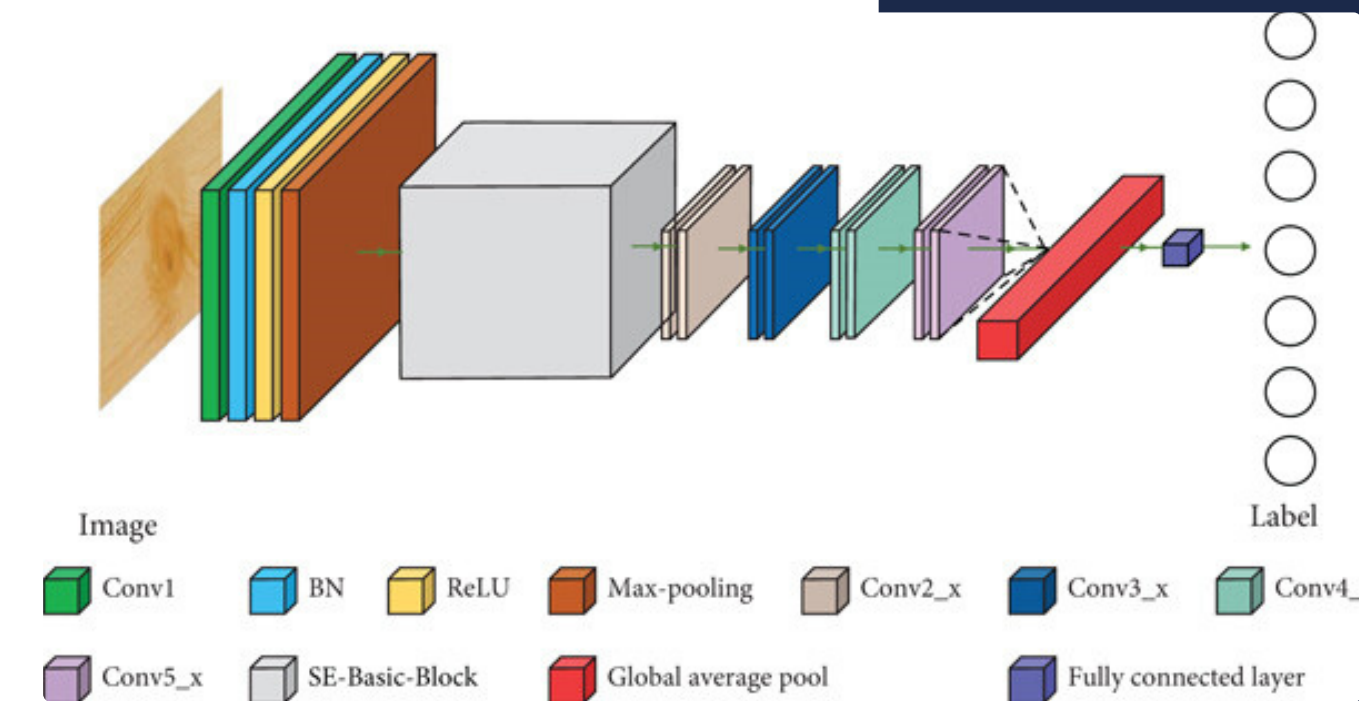
- **16 Classes**: CLEAN + 5 jamming types (LN, LWF, TICK, TRI, TRIW) × 3 power levels
- **LOW JSR**: +1, +4, +9 dB, **MID JSR**: +14, +19, +24 dB, **HIGH JSR**: +29, +34 dB

3) Input & Training

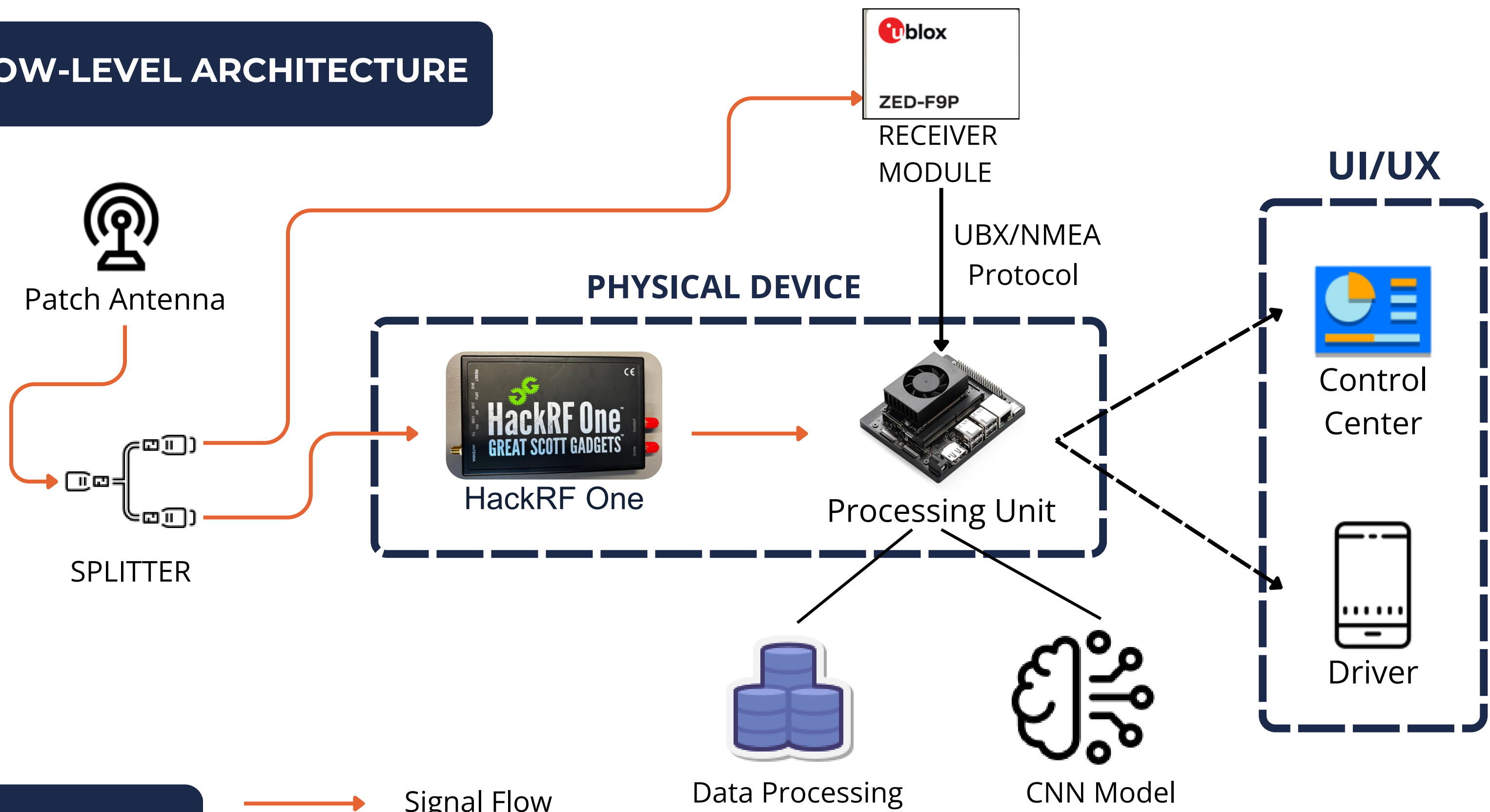
- **STFT spectrograms** (int8 numpy arrays, single-channel)
- **Split**: 50% train, 15% val, 35% test | **Batch size**: 128

Satisfied requirements:

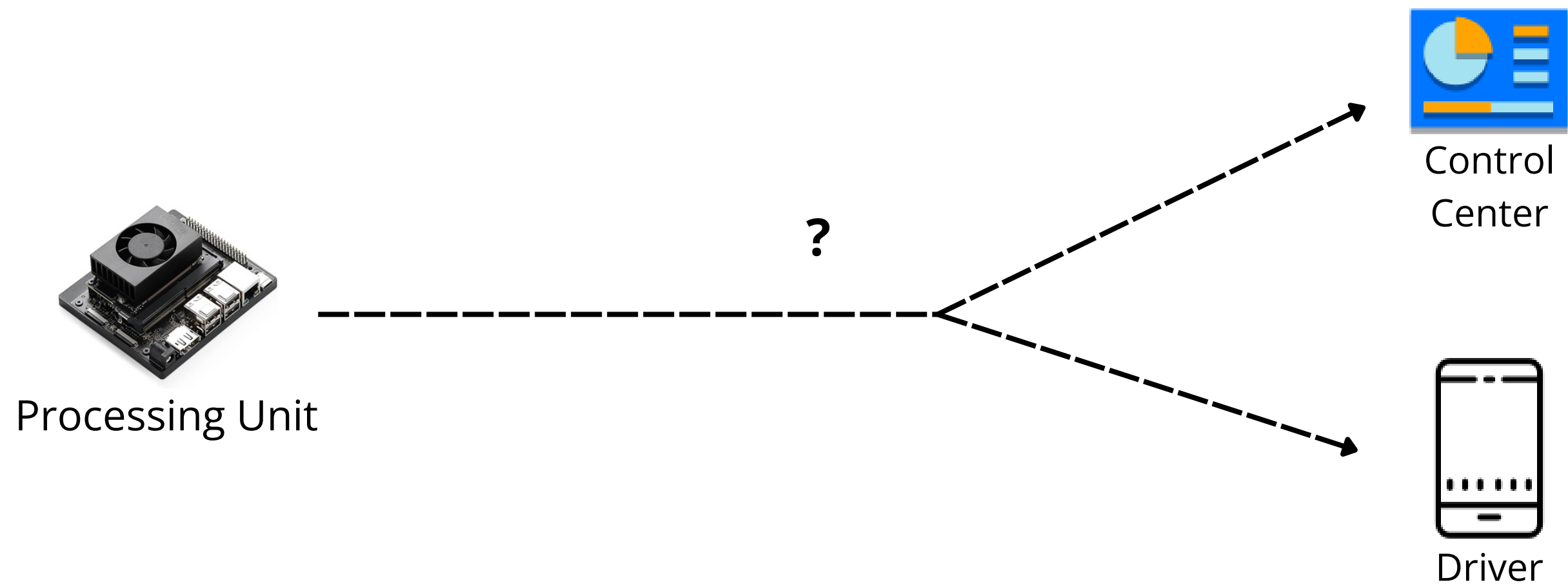
UREQ_1 , UREQ_2 , UREQ_3 , FREQ_1 , FREQ_2 , FREQ_3 , FREQ_4 , FREQ_5 , TREQ_1 , TREQ_2 ,
TREQ_3 , TREQ_4 , TREQ_5 , TREQ_6 , TREQ_7 , TREQ_8 , TREQ_9



LOW-LEVEL ARCHITECTURE



COMMUNICATION



1) Network Resilience

- Trucks go through tunnels, rural areas, and underground parking
- HTTP: requires full TCP handshake for every packet so fails in zones with poor coverage
- MQTT: maintains lightweight session and auto-reconnects when signal returns

2) Quality of Service (QoS) Levels

- QoS 0: "Fire and forget" for periodic status updates
- QoS 1: "At least once" for Jamming alerts (retry until confirmed)
- QoS 2: "Exactly once"

3) Last Will & Testament (LWT)

- If device loses power/connection
- MQTT broker automatically publishes predefined "DEVICE OFFLINE" message
- Control center immediately knows truck is compromised

Satisfied requirements:

UREQ_1 , UREQ_2 , UREQ_3 , FREQ_1 , FREQ_2 , FREQ_3 , FREQ_4 , FREQ_5 , TREQ_1 , TREQ_2 ,
TREQ_3 , TREQ_4 , TREQ_5 , TREQ_6 , TREQ_7 , TREQ_8 , TREQ_9

BROKER

Problem: modern web browsers (Chrome, Safari, Edge) are strictly designed for security. They don't allow web pages to open raw TCP sockets. If we try to connect the UI directly to the standard MQTT port (8883), the browser would block it.

Solution: we wrap the MQTT message inside a WebSocket envelope so that the UI can receive it using HiveMQ that bridges this gap

HiveMQ acts as a Translator:

- The Truck sends a packet via TCP to port 8883.
- HiveMQ receives the TCP packet. It removes the TCP headers to expose the raw MQTT payload (JSON data).
- HiveMQ checks who is subscribed. It sees the UIs are subscribed but are connected via WebSockets.
- HiveMQ takes that same MQTT payload, wraps it inside a WebSocket Frame, and pushes it out port 8884 to the Browser

Advantages:

- No need to maintain our own MQTT broker + realistic scenario
- TLS/SSL encryption + username/password authentication
- Handles multiple vehicles without infrastructure changes



COMMUNICATION

PUBLISHER to:

- NMON/alert/vehicle_id
- NMON/status/vehicle_id



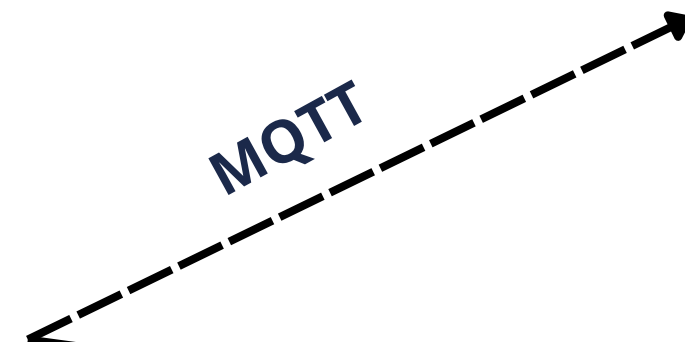
Processing Unit

MQTT



HiveMQ
(Cloud Broker)

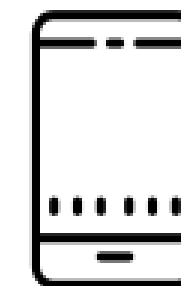
MQTT



MQTT



Control
Center



Driver

SUBSCRIBER to:

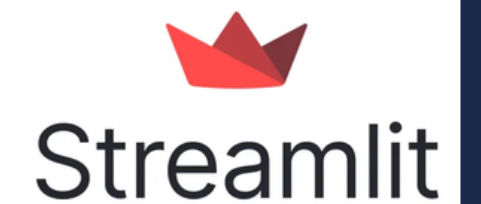
- NMON/alert/#
- NMON/status/#

SUBSCRIBER to:

- NMON/alert/vehicle_id

vehicle_id is a unique identifier of the vehicle our device is mounted on. In this architecture it's the plate

USER INTERFACE



Control Center UI

- **Technology:** Streamlit (Python) , a framework specifically designed for rapidly building data-focused web applications. It's popular in the data science and machine learning communities because it allows to create interactive dashboards with minimal code
- **Features:**
 - Fast development
 - Provides a real web server (Tornado) that can be deployed like any production application
 - Designed for analytics, includes built-in components for metrics, charts, and real-time data visualization
 - Can use the more efficient native MQTT protocol over TCP (port 8883)

Mobile UI

- **Technology:** HTML/CSS + MQTT.js
- **Features:**
 - Simplicity , no installation required ,works on any phone/tablet
 - Web browsers can't open raw TCP,but can open WebSocket for receiving real-time alerts via HiveMQ (port 8884)

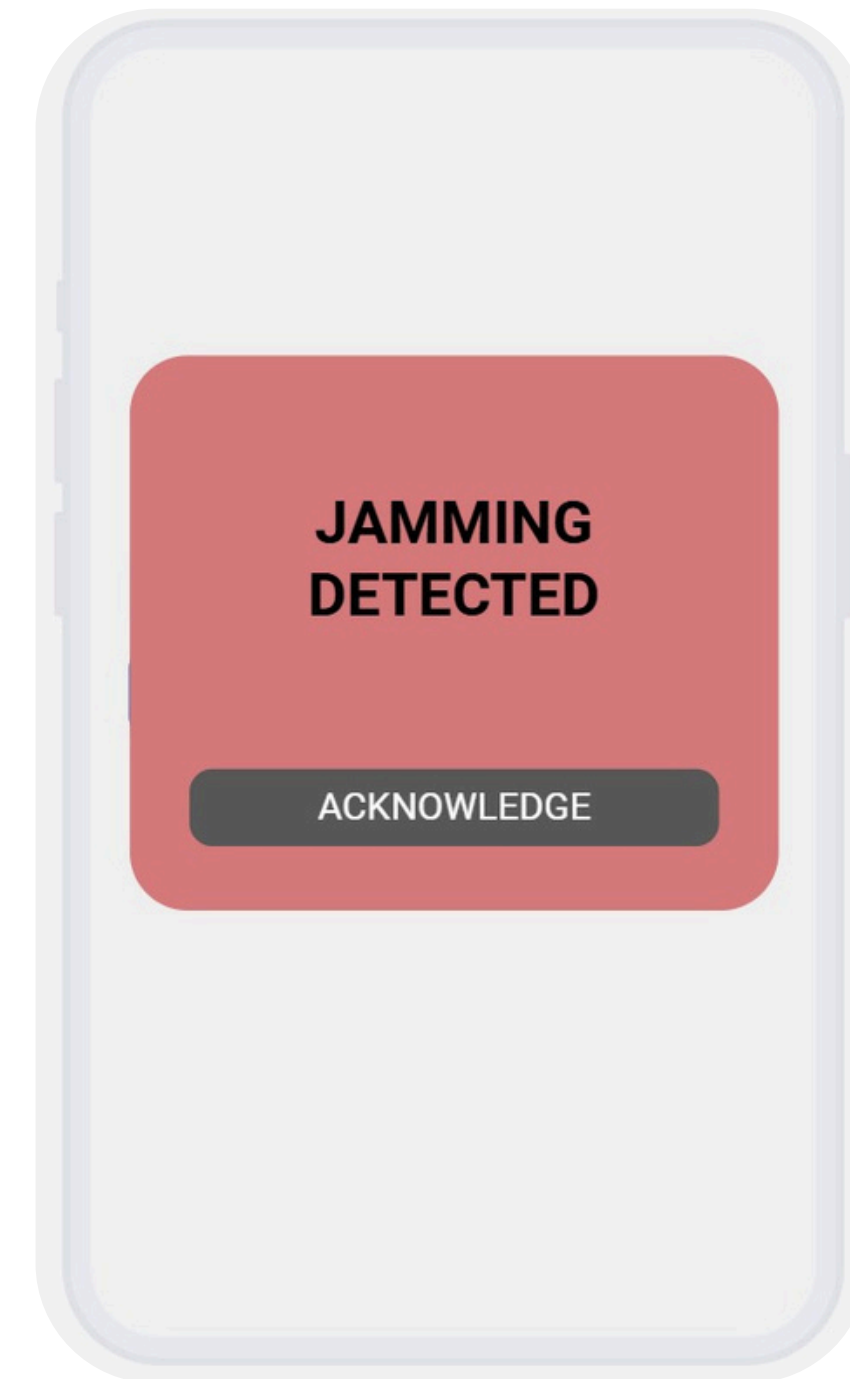
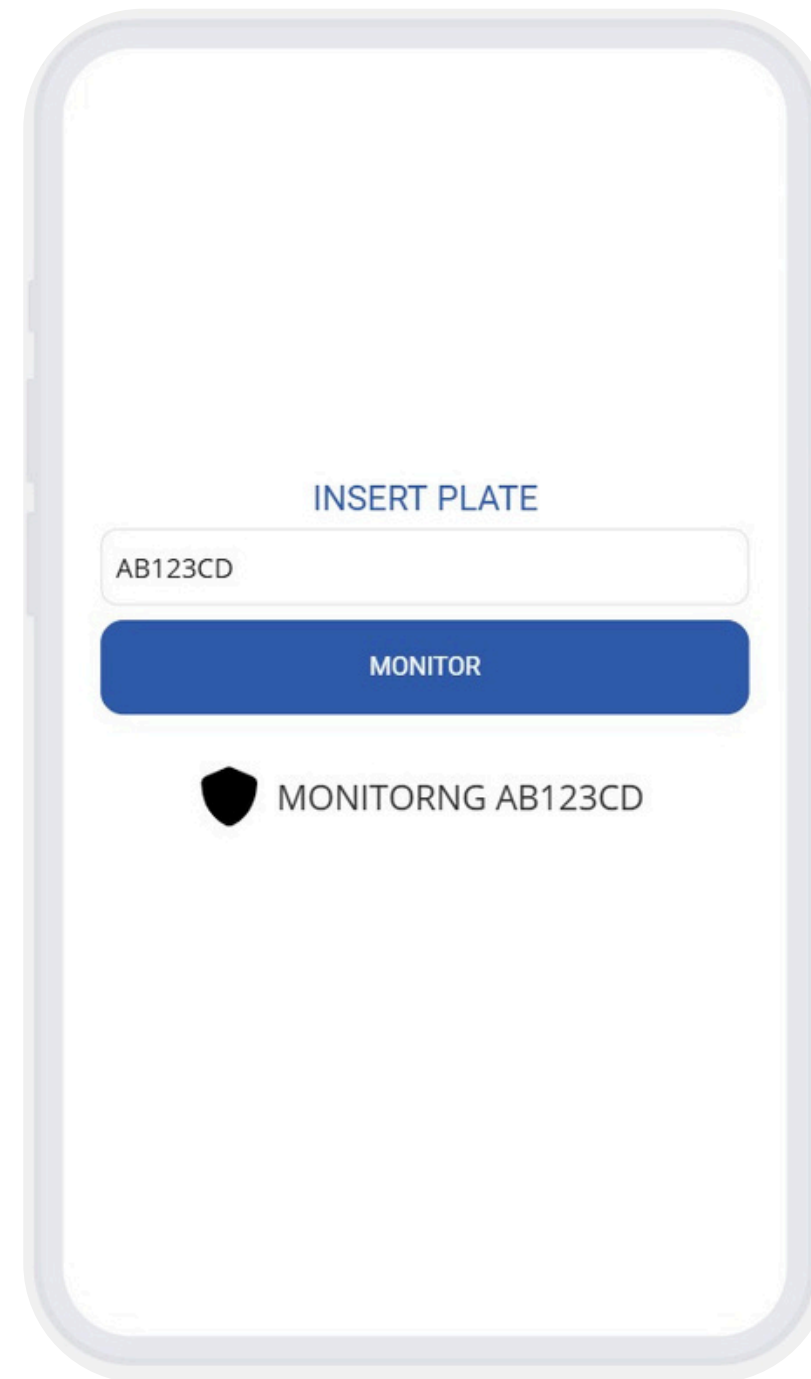
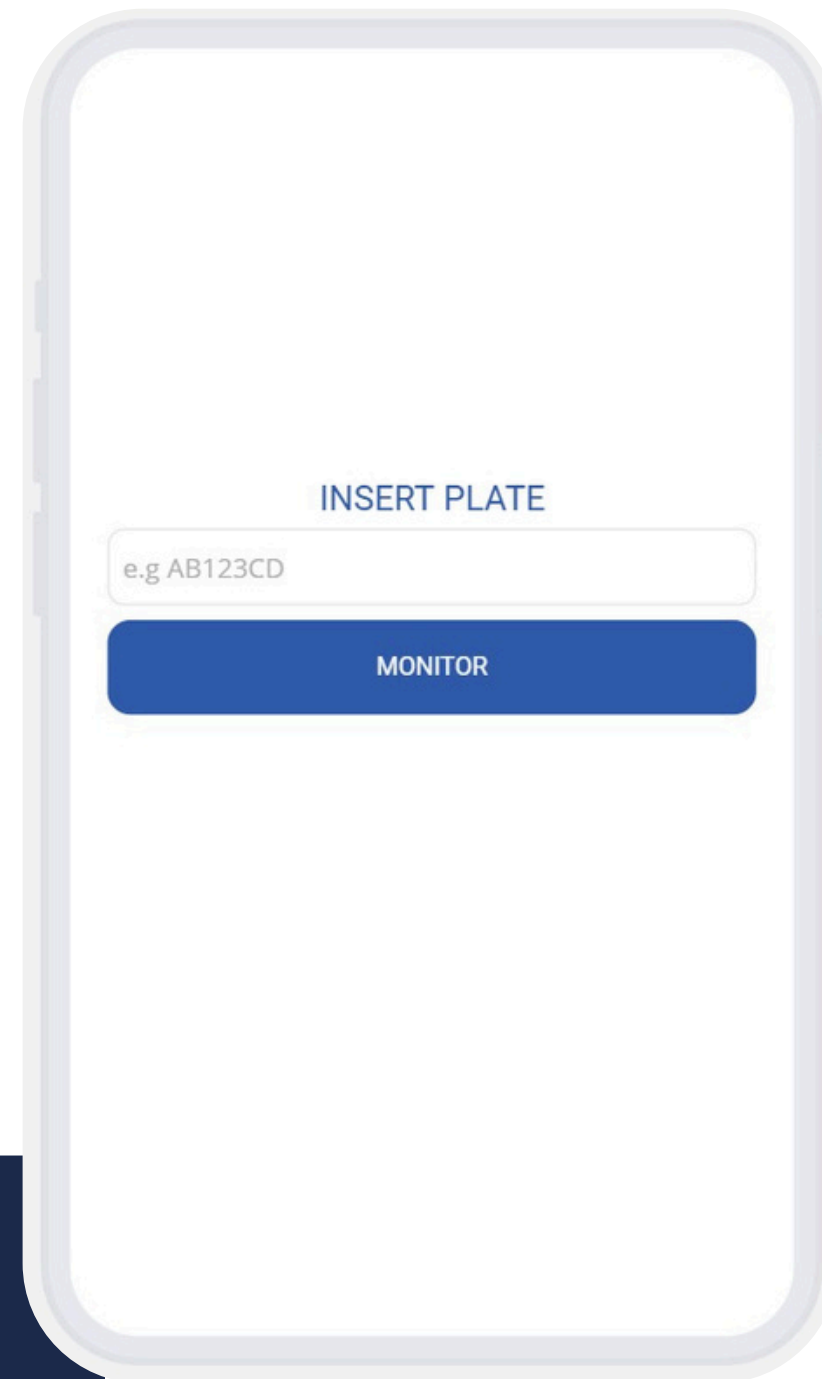


Satisfied requirements:

UREQ_1 , UREQ_2 , UREQ_3 , FREQ_1 , FREQ_2 , FREQ_3 , FREQ_4 , FREQ_5 , TREQ_1 , TREQ_2 ,
TREQ_3 , TREQ_4 , TREQ_5 , TREQ_6 , TREQ_7 , TREQ_8 , TREQ_9

USER INTERFACE

The only action performed by the driver to is insert the plate



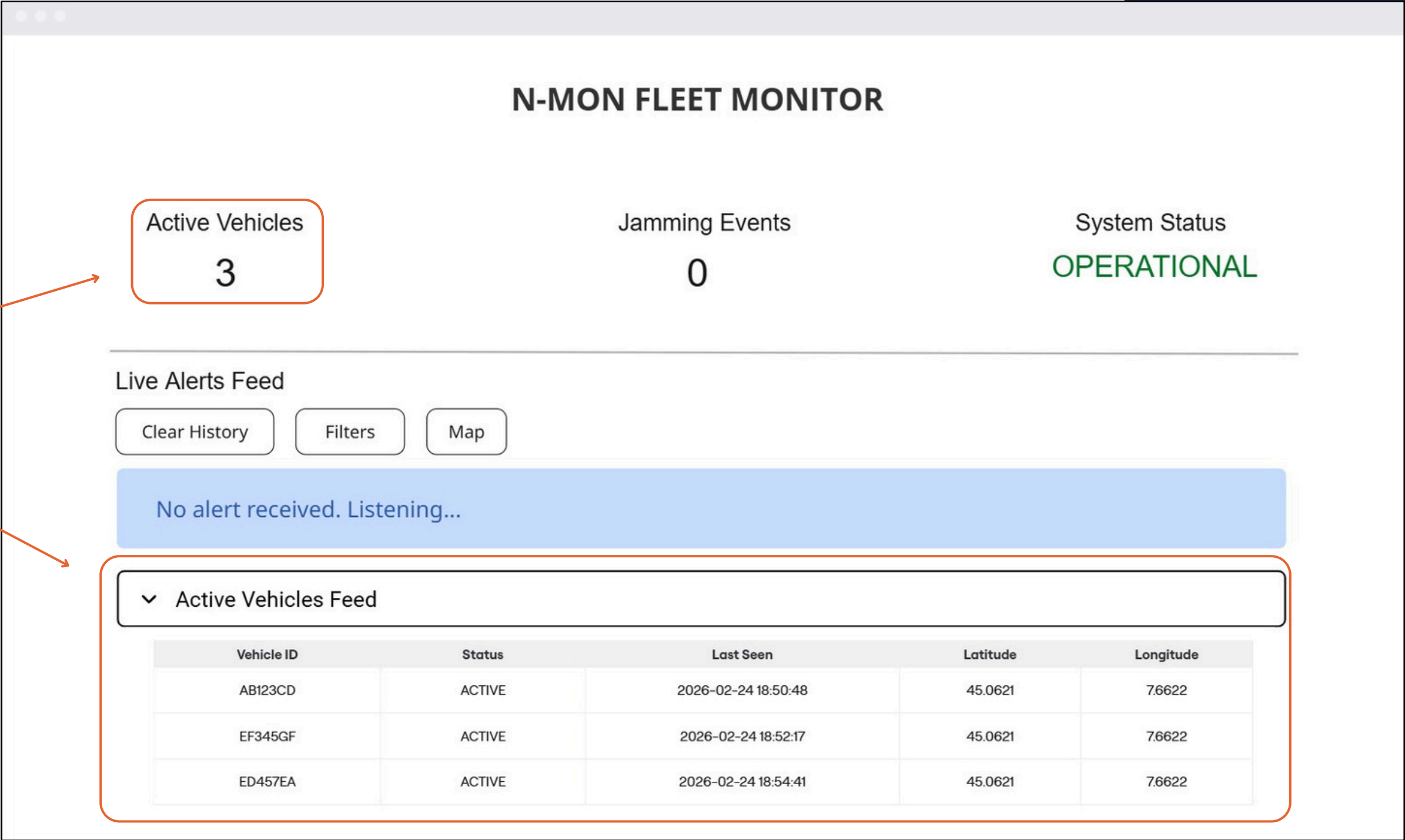
USER INTERFACE

Normal Operation Mode:

When no jamming event is detected, the interface displays active vehicle information :

- Plate
- Last Seen
- Position
- Active Vehicles Counter

while the jamming event counter remains at 0



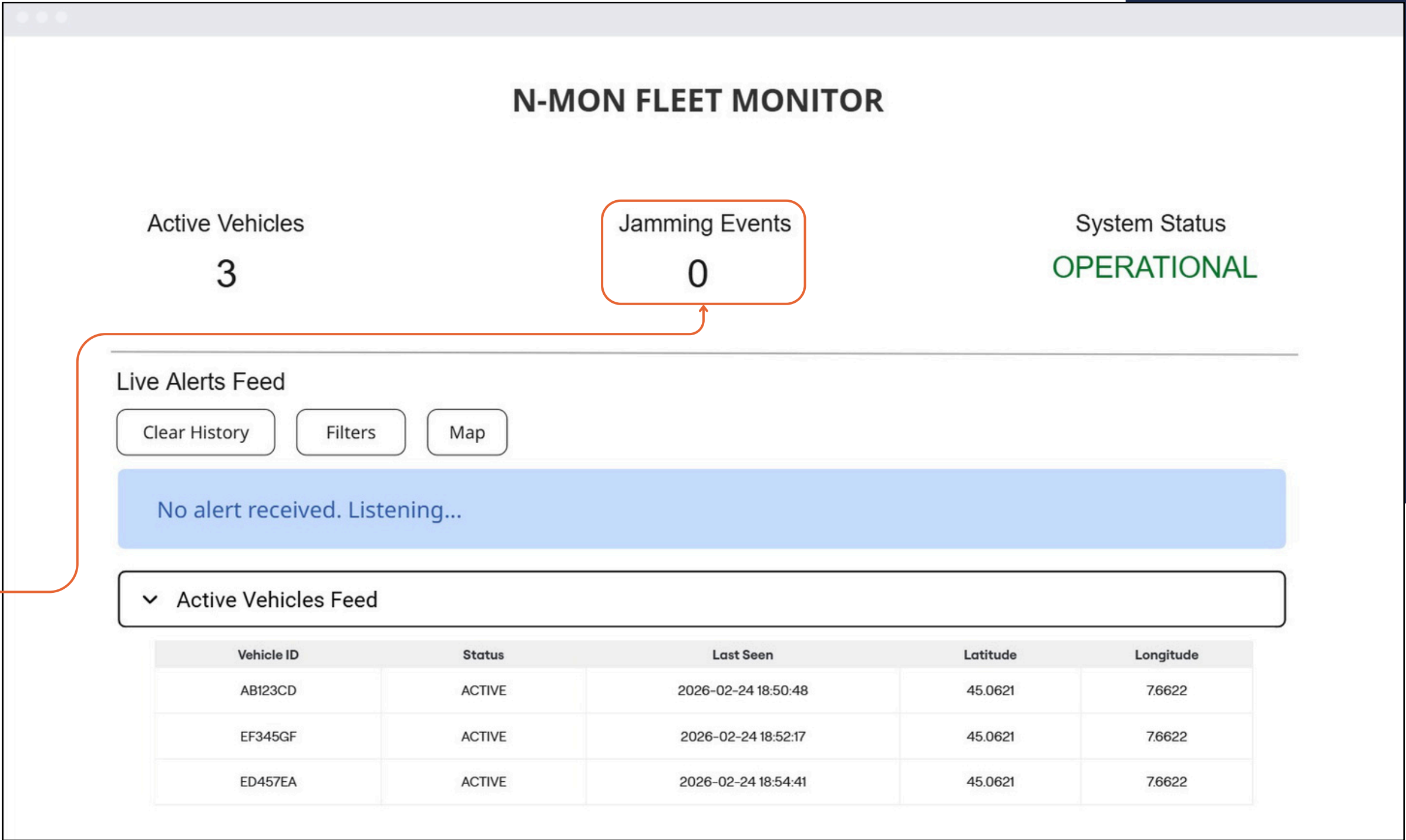
USER INTERFACE

Normal Operation Mode:

When no jamming event is detected, the interface displays active vehicle information :

- Plate
- Last Seen
- Position
- Active Vehicles Counter

while the jamming event counter remains at 0

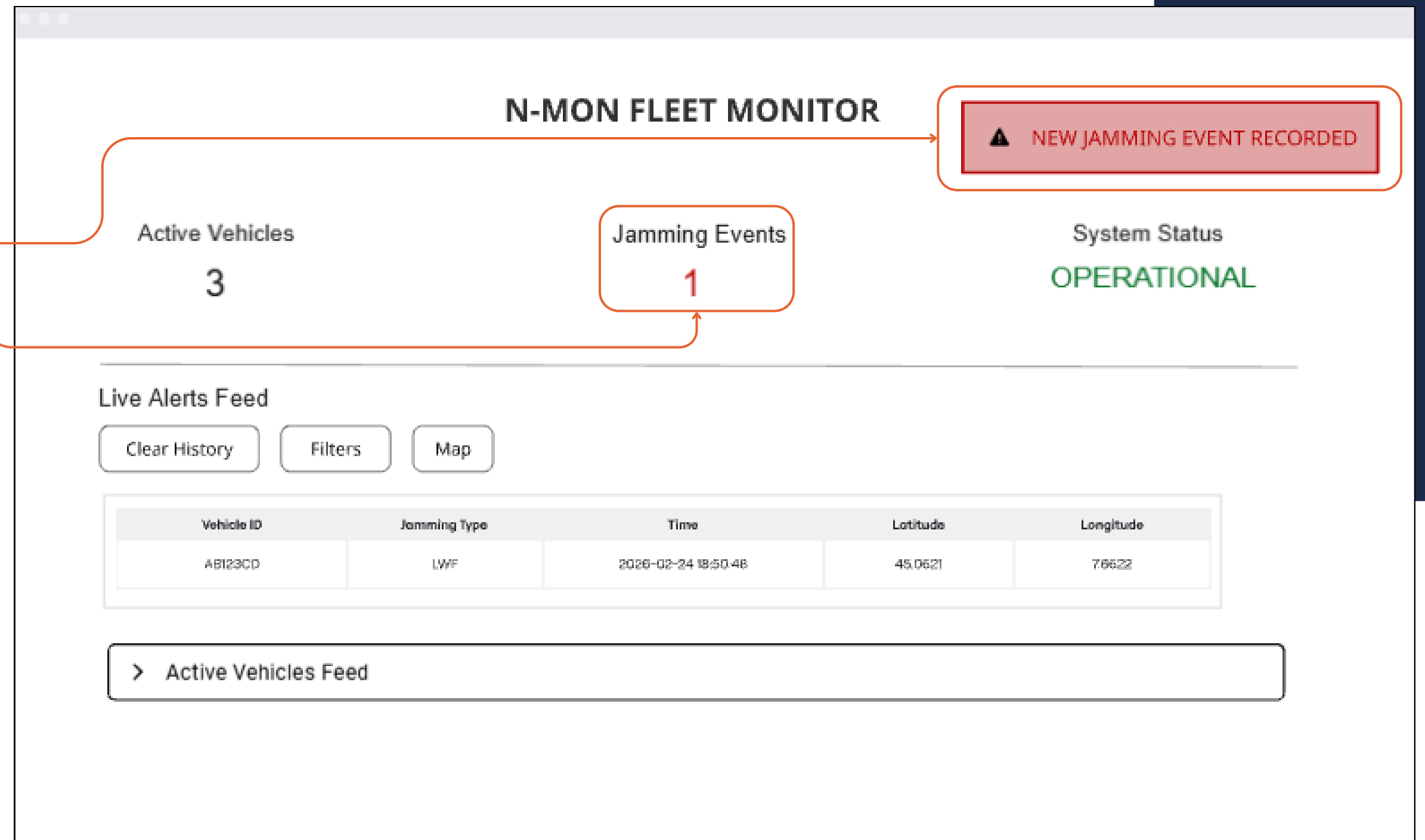


USER INTERFACE

Jamming Event Detected:

When a jamming event occurs, the jamming counter increases, a pop-up alert is triggered, and the jamming information table is automatically populated with the detected event details:

- Plate
- Jamming Type
- Time
- Position



USER INTERFACE

Jamming Event Detected:

When a jamming event occurs, the jamming counter increases, a pop-up alert is triggered, and the jamming information table is automatically populated with the detected event details:

- Plate
- Jamming Type
- Time
- Position

The screenshot displays the 'N-MON FLEET MONITOR' dashboard. At the top right, a red alert box states 'NEW JAMMING EVENT RECORDED'. The dashboard features three main metrics: 'Active Vehicles' with a count of 3, 'Jamming Events' with a count of 1, and 'System Status' which is 'OPERATIONAL'. Below these metrics is a 'Live Alerts Feed' section containing buttons for 'Clear History', 'Filters', and 'Map'. A table within this section lists the detected jamming event. An orange arrow points from the 'Jamming Type' column of this table to the 'Jamming Type' bullet point in the text on the left. At the bottom of the dashboard, there is a button labeled '> Active Vehicles Feed'.

Vehicle ID	Jamming Type	Time	Latitude	Longitude
AB123CD	LWF	2026-02-24 18:50:46	45.0621	7.6622

USER INTERFACE

User Interaction Features:

The operator can apply filters via the *Filter* button or visualize events through the *Map* button.

The Filter menu allows filtering by:

- Plate
- Jamming Type
- Time Range

The Map displays the locations of jamming events on an interactive view, reflecting the filters currently applied

N-MON FLEET MONITOR

Active Vehicles
3

Jamming Events
1

System Status
OPERATIONAL

Live Alerts Feed

Clear History

Filters

Map

Filter Options

Vehicle ID

Choose Options

Alert Type

Choose Options

Time From (YYYY-MM-DD HH:MM:SS)

e.g. 2026-01-01 08:00:00

Time To (YYYY-MM-DD HH:MM:SS)

e.g. 2026-12-31 23:59:59

Vehicle ID	Jamming Type	Time	Latitude	Longitude
AB123CD	LWF	2026-02-24 18:50:48	45.0621	7.6622

>

Active Vehicles Feed

USER INTERFACE

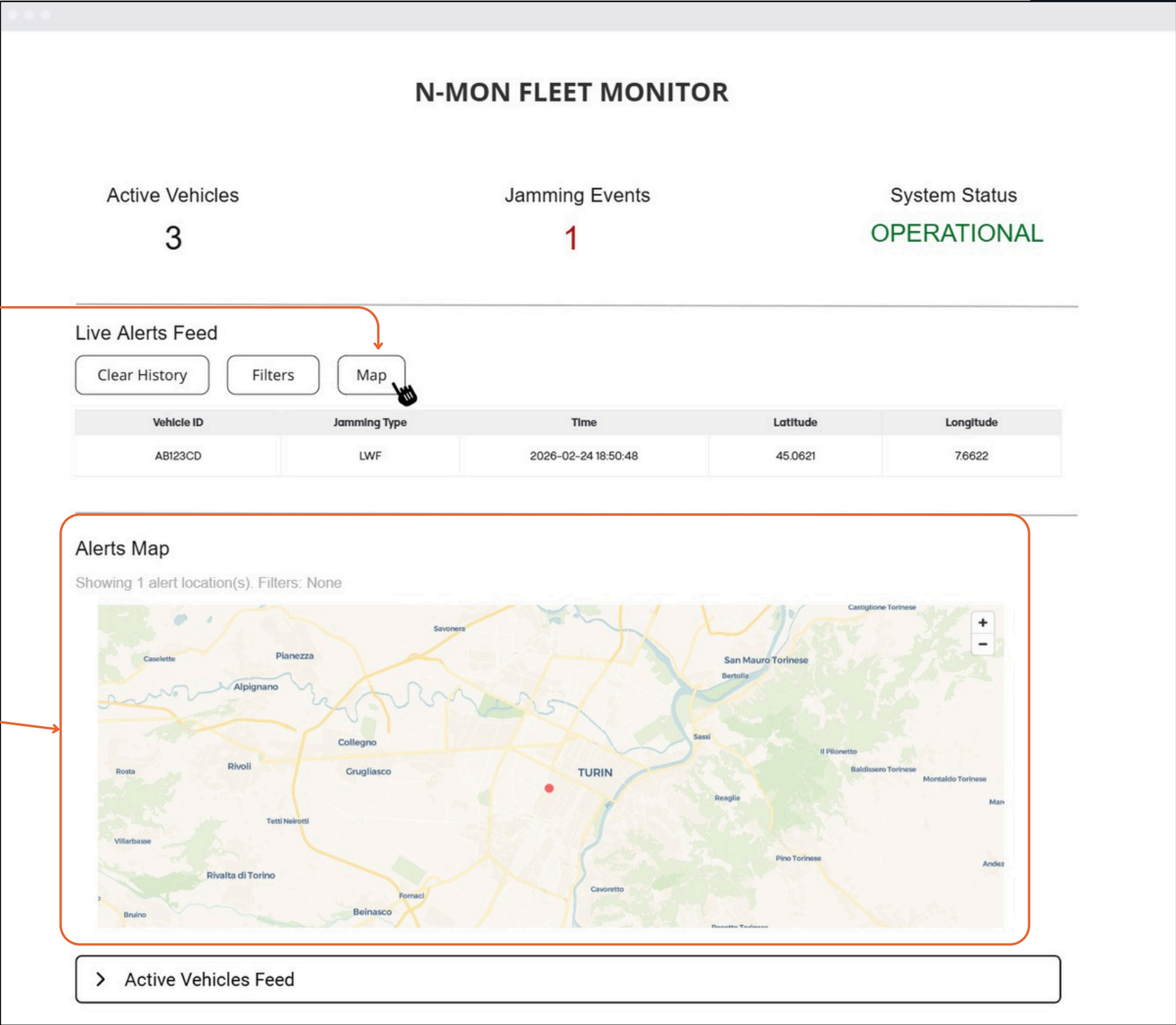
User Interaction Features:

The operator can apply filters via the *Filter* button or visualize events through the *Map* button.

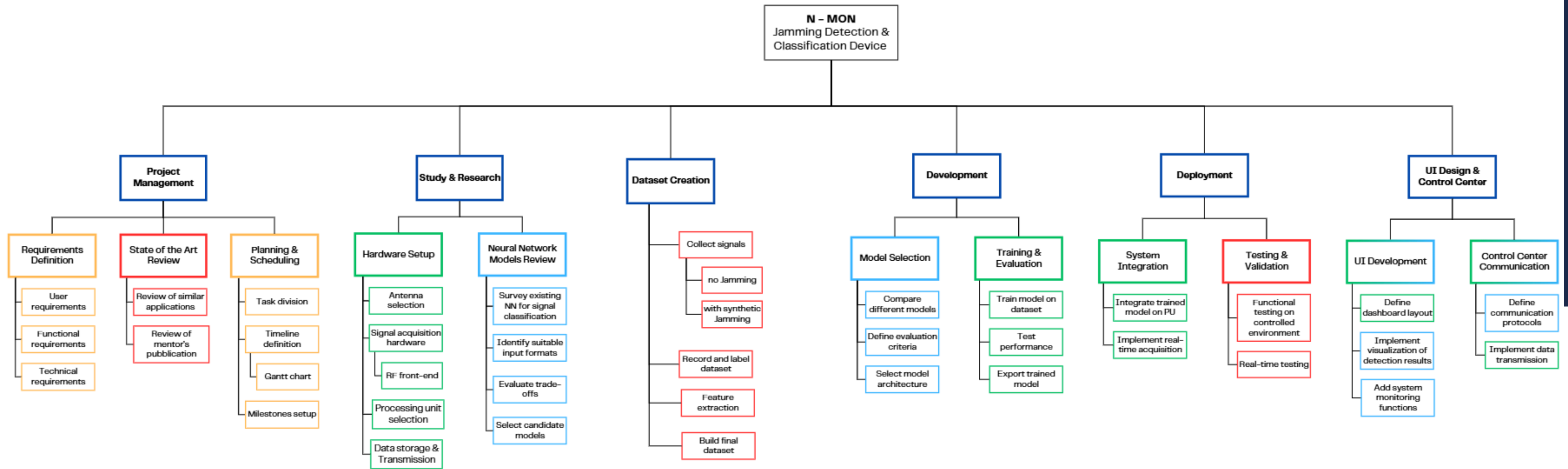
The Filter menu allows filtering by:

- Plate
- Jamming Type
- Time Range

The Map displays the locations of jamming events on an interactive view, reflecting the filters currently applied

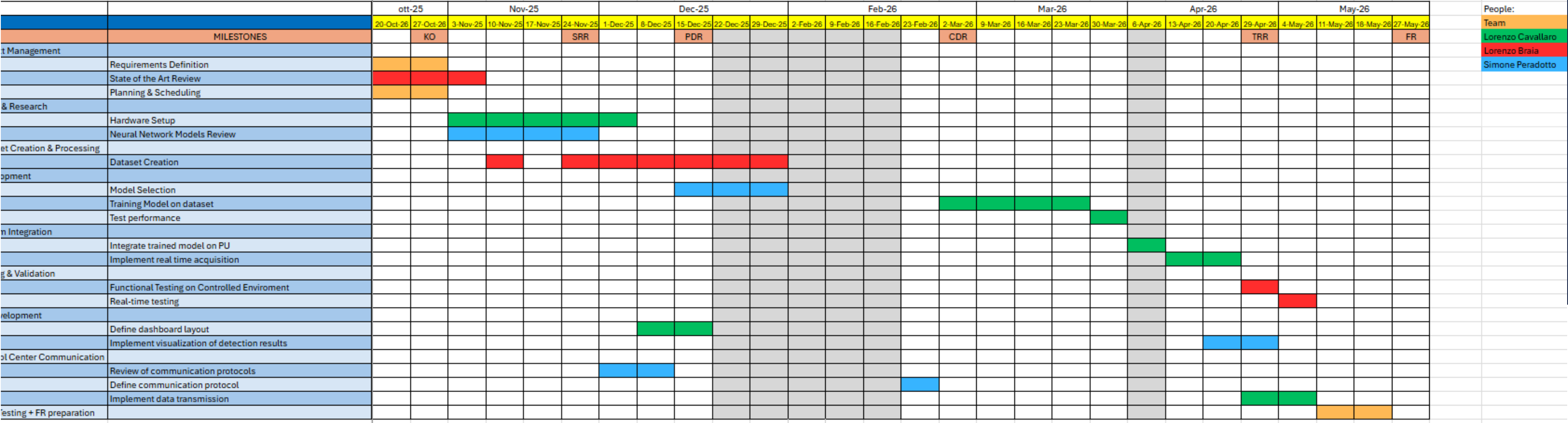


WBS



<https://www.canva.com/design/DAG14EVzFeY/AZdbRfanr0YUQmWhZWVmg/edit>

GANTT CHART



Link accessible with Polito credentials :

https://politoit-my.sharepoint.com/:x:/g/personal/s343420_studentsi_polito_it/EVHoF8byxhZOuSA_gHzWz6ABPUtiU81AzwQXv3SZYcAgyA?e=vecgTL



THANK YOU!

For your attention